



AP
Taller 3

Tomas Parra

Teoría de lenguajes y laboratorio

Profesora: Luz Páez

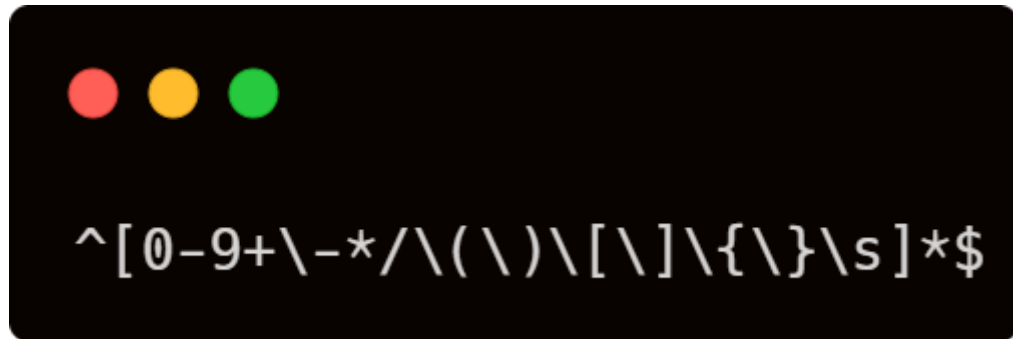
Medellín, 2026

Facultad de Ingeniería
Universidad de Antioquia

1. Diseño de la solución

1.1. Nivel 1: Validación

Se utiliza la librería estándar <regex> de C++ con el siguiente patrón, que reconoce únicamente números, operadores (+ - * /), símbolos de agrupación (([] { }) y espacios en blanco:



Si la expresión completa no coincide con el patrón, el programa recorre la cadena carácter por carácter para identificar la posición exacta y el símbolo que no pertenece al conjunto permitido, por ejemplo '@' o '#'.

1.2. Nivel 2: Validación con AP

Una vez superada la validación de caracteres, el programa simula el comportamiento de un autómata de pila para comprobar el balanceo de los símbolos de agrupación:

- Al encontrar un símbolo de apertura (([{), se apila junto con su posición.
- Al encontrar un símbolo de cierre, se compara con el símbolo en el tope de la pila mediante la función coinciden(). Si no corresponde, se reporta el símbolo esperado, el símbolo encontrado y la posición.
- Si aparece un cierre sin apertura previa (pila vacía), se reporta como cierre inesperado.
- Si al finalizar el recorrido quedan símbolos sin cerrar en la pila, se reporta cuál apertura quedó pendiente y en qué posición.

Este mecanismo reproduce exactamente el comportamiento pedido en el enunciado: para la expresión {(25 + 8] * 10}, el sistema detecta que el símbolo '(' fue cerrado incorrectamente con ']' en la posición 9.

1.3. Nivel 3: Validación de la estructura matemática

Superados los dos niveles anteriores, el programa tokeniza la expresión (agrupando dígitos consecutivos en un solo número) y recorre la secuencia de tokens verificando combinaciones válidas entre un token y el siguiente:

- Después de un NÚMERO o un CIERRE solo puede venir un OPERADOR o un CIERRE.
- Después de un OPERADOR o una APERTURA solo puede venir un NÚMERO o una APERTURA.
- La expresión no puede iniciar ni terminar con un OPERADOR.

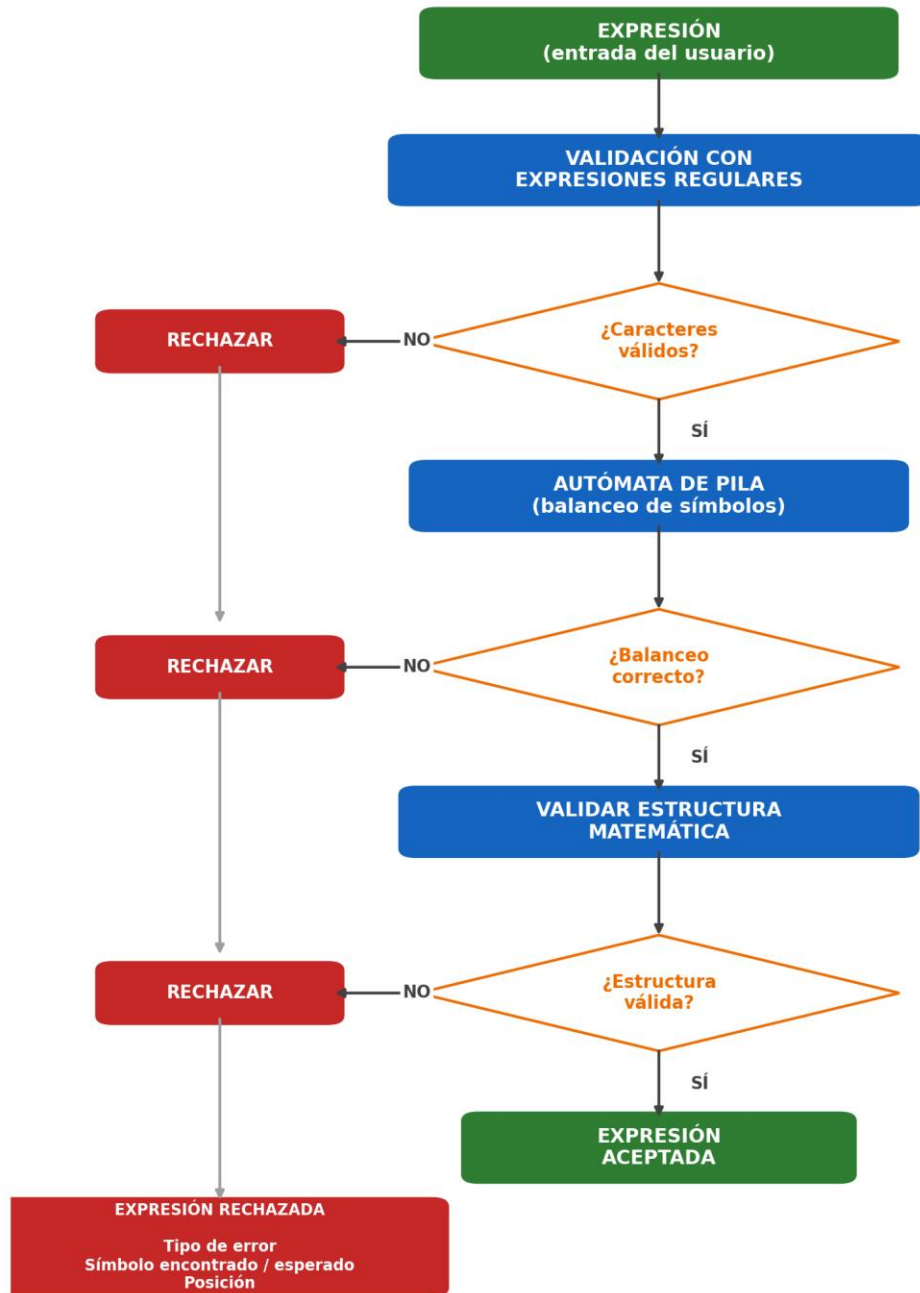
Esta lógica detecta los casos solicitados: dos operadores consecutivos ($25 ++ 8$), operador al inicio ($+ 25 * 8$), operador al final ($25 + 8 *$), y dos números consecutivos sin operador ($25 8 + 3$).

1.4. Flujo general

Las tres validaciones se ejecutan en cascada: si un nivel falla, el programa rechaza la expresión de inmediato sin continuar con los niveles siguientes, tal como lo indica el diagrama de flujo del enunciado.

2. Diagrama de flujo

Flujo del Validador de Expresiones Matemáticas



3. Funciones implementadas

Función	Responsabilidad
validarCaracteres(expresion)	Aplica la expresión regular que reconoce el conjunto de caracteres permitido (dígitos, operadores, agrupadores, espacios) y localiza el primer carácter inválido si existe.
esNumero / esOperador / esApertura / esCierre / esEspacio	Funciones de clasificación de un carácter individual, usadas por los tres niveles de validación.
Coinciden(apertura, cierre)	Indica si un símbolo de cierre corresponde al símbolo de apertura dado.
validarBalanceo(expresion)	Simula el autómata de pila: apila cada apertura y, al llegar un cierre, valida contra el tope de la pila. Reporta símbolo esperado/encontrado y posición.
tokenizar(expresion)	Convierte la cadena en una lista de tokens (NUMERO, OPERADOR, APERTURA, CIERRE) agrupando dígitos consecutivos en un solo número.
validarEstructura(expresion)	Recorre los tokens verificando la secuencia válida (número/cierre → operador/cierre; operador/apertura → número/apertura), detectando operador al inicio, al final, operadores y números consecutivos.
mostrarResultado(...)	Imprime el reporte final: estado de cada nivel de validación y, si la expresión fue rechazada, tipo de error, símbolo encontrado, símbolo esperado y posición.
procesarExpresion(expresion)	Orquesta la validación en cascada: si un nivel falla, detiene el proceso y no continúa con los siguientes (tal como exige el flujo del taller).

4. Casos de prueba

Expresion	Resultado	Nivel que falla	Detalle
(25 + 8) @ [10 - 3]	RECHAZADA	Caracteres	@ no permitido, posición 10
(25 + 8) # 3	RECHAZADA	Caracteres	# no permitido, posición 10
{(25 + 8) * [10 - 3]}		---	Pasa los tres niveles
{(25 + 8) * [10 - 3]}	RECHAZADA	Balanceo	Se esperaba ')' y se encontró ']', posición 9
25 ++ 8	RECHAZADA	Estructura	Dos operadores consecutivos, posición 5
10 * / 2	RECHAZADA	Estructura	Dos operadores consecutivos, posición 6
+ 25 * 8	RECHAZADA	Estructura	Operador al inicio, posición 1
25 + 8 *	RECHAZADA	Estructura	Operador al final, posición 8
25 8 + 3	RECHAZADA	Estructura	Dos números consecutivos sin operador, posición 4
25 + 8 * 3		---	Pasa los tres niveles
{(25 + 8) * 10}	RECHAZADA	Balanceo	Se esperaba ')' y se encontró ']', posición 9

5. Formato de salida

Ante una expresión rechazada, el programa reporta los cuatro datos solicitados por la empresa: tipo de error, símbolo que lo produjo, posición dentro de la expresión y la validación que falló. Ejemplo para la entrada $\{(25 + 8] * 10\}$:



```
ERROR EN LA POSICION: 9
```

```
TIPO DE ERROR:
```

```
Simbolos de agrupacion incorrectamente balanceados
```

```
SIMBOLO ENCONTRADO:
```

```
]
```

```
SIMBOLO ESPERADO:
```

```
)
```

6. Pruebas



```
=====
VALIDADOR DE EXPRESIONES MATEMATICAS AVANZADAS
(Regex + Automata de Pila + Validacion Estructural)
=====
```

```
Escriba 'salir' para terminar el programa.
```

```
Ingrese una expresion: 1*2
```

```
=====
ANALIZADOR DE EXPRESIONES
=====
```

```
Expresion ingresada:
```

```
1*2
```

```
Validacion de caracteres: VALIDA
```

```
Validacion de simbolos:  VALIDA
```

```
Validacion de estructura: VALIDA
```

```
RESULTADO FINAL:
```

```
EXPRESION ACEPTADA
```

```
-----
```



```
=====
VALIDADOR DE EXPRESIONES MATEMATICAS AVANZADAS
(Regex + Automata de Pila + Validacion Estructural)
=====
Escriba 'salir' para terminar el programa.

Ingrese una expresion: (1)(2)

=====
ANALIZADOR DE EXPRESIONES
=====

Expresion ingresada:
(1)(2)

Validacion de caracteres: VALIDA
Validacion de simbolos:  VALIDA
Validacion de estructura: VALIDA

RESULTADO FINAL:
EXPRESION ACEPTADA

-----
```



```
=====
VALIDADOR DE EXPRESIONES MATEMATICAS AVANZADAS
(Regex + Automata de Pila + Validacion Estructural)
=====
Escriba 'salir' para terminar el programa.

Ingrese una expresion: (1(2)[3])

=====
ANALIZADOR DE EXPRESIONES
=====

Expresion ingresada:
(1(2)[3])

Validacion de caracteres: VALIDA
Validacion de simbolos:  VALIDA
Validacion de estructura: VALIDA

RESULTADO FINAL:
EXPRESION ACEPTADA

-----
```

```
=====
VALIDADOR DE EXPRESIONES MATEMATICAS AVANZADAS
(Regex + Automata de Pila + Validacion Estructural)
=====
Escriba 'salir' para terminar el programa.

Ingrese una expresion: (1)(2]

=====
ANALIZADOR DE EXPRESIONES
=====

Expresion ingresada:
(1)(2]

Validacion de caracteres: VALIDA
Validacion de simbolos:  INVALIDA

RESULTADO FINAL:
EXPRESION RECHAZADA

ERROR EN LA POSICION: 6

TIPO DE ERROR:
Simbolos de agrupacion incorrectamente balanceados

SIMBOLO ENCONTRADO:
]

SIMBOLO ESPERADO:
)

=====
```

7. Conclusiones

- Combinar Expresiones Regulares con un Autómata de Pila permite separar responsabilidades: el regex resuelve validaciones léxicas (qué caracteres son válidos) y la pila resuelve validaciones de estructura anidada (balanceo de símbolos), que un regex por sí solo no puede expresar.
- El diseño modular (una función por responsabilidad) facilita probar cada nivel de validación de forma independiente y localizar errores con precisión (tipo, símbolo y posición).
- La validación en cascada evita procesamiento innecesario: si la expresión falla en un nivel temprano, no se ejecutan los niveles posteriores.